

RAG Pipeline

Technical Design Document

1. High-Level Design (HLD)

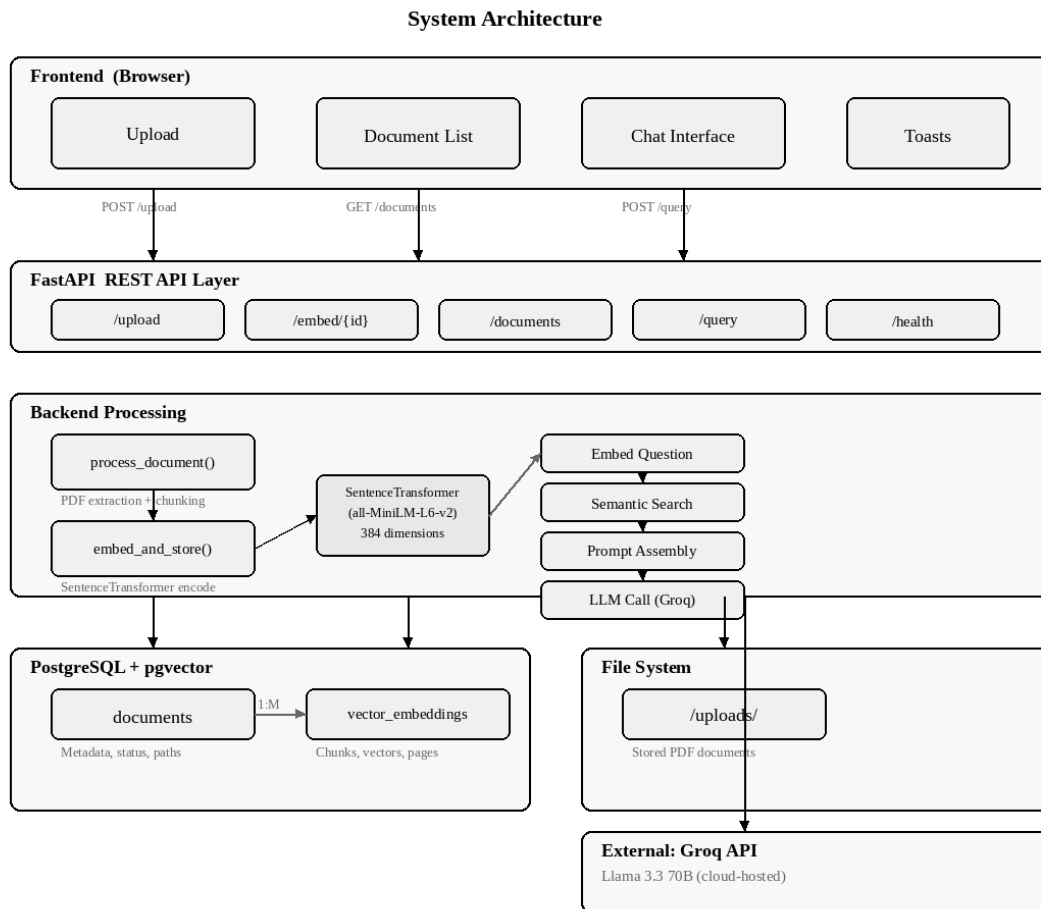
1.1 System Overview

I have designed a Retrieval-Augmented Generation (RAG) application that enables users to upload PDF documents, create vector embeddings, and query the documents through a conversational chat interface powered by a Large Language Model (LLM).

The system follows a three-stage pipeline:

- **Ingestion:** PDF upload, text extraction, chunking, and vector embedding generation
- **Storage:** Document metadata in PostgreSQL, vector embeddings in pgvector
- **Retrieval & Generation:** Semantic similarity search, context assembly, and LLM-powered answer generation

1.2 Architecture Diagram



1.3 Tech Stack

Layer	Technology	Purpose
Language	Python 3.11+	Backend logic, ML model integration
Backend	FastAPI	Async REST API with auto-generated Swagger docs
Embedding	all-MiniLM-L6-v2	384-dim sentence embeddings, runs locally
Vector DB	PostgreSQL + pgvector	Cosine similarity search on embeddings
LLM	Groq (Llama 3.3 70B)	Free API, fast inference for answer generation

PDF Processing	pdfplumber	Text extraction from PDF documents
Frontend	HTML / CSS / JS	Single-page interface with drag-drop upload
Validation	Pydantic	Request/response model validation

1.4 External Integrations

- **Groq API:** Cloud-hosted LLM inference. Requires API key via GROQ_API_KEY environment variable.
- **SentenceTransformers:** Model downloaded once at startup; runs locally with no external calls during inference.
- No other external services required. The system is fully self-contained with a local PostgreSQL instance.

2. Low-Level Design (LLD)

2.1 Core Functions

Function 1: process_document(file_path)

Responsibility: Extract text from a PDF and split it into overlapping chunks.

- **Input:** file_path (str) — path to the PDF file on disk
- **Output:** List of dictionaries, each containing "page_no" (int) and "text" (str)

Process

- Open the PDF using pdfplumber
- Iterate through each page using enumerate for page numbers
- Extract text; skip pages that return None
- Normalize whitespace: collapse tabs, newlines, multiple spaces into single spaces
- Chunk the text using a sliding window: step 400 chars, window 500 chars (100-char overlap)
- Append each chunk with its 1-indexed page number to the output list

Function 2: embed_and_store(document_id, chunks)

Responsibility: Convert text chunks to vectors and persist them in pgvector.

- **Input:** document_id (int), chunks (list of dicts from Function 1)
- **Output:** None (side effect: rows inserted into vector_embeddings table)

Process

- Open a database connection with pgvector type registered
- For each chunk, encode the text using SentenceTransformer.encode()
- Convert the numpy array to a Python list with .tolist()
- Execute an INSERT with document_id, page_no, text, and vector
- Commit all inserts in a single transaction after the loop

Function 3: query(question, document_ids)

Responsibility: Full RAG pipeline — embed query, retrieve context, generate answer.

- **Input:** question (str), document_ids (list of int)
- **Output:** Answer string from the LLM

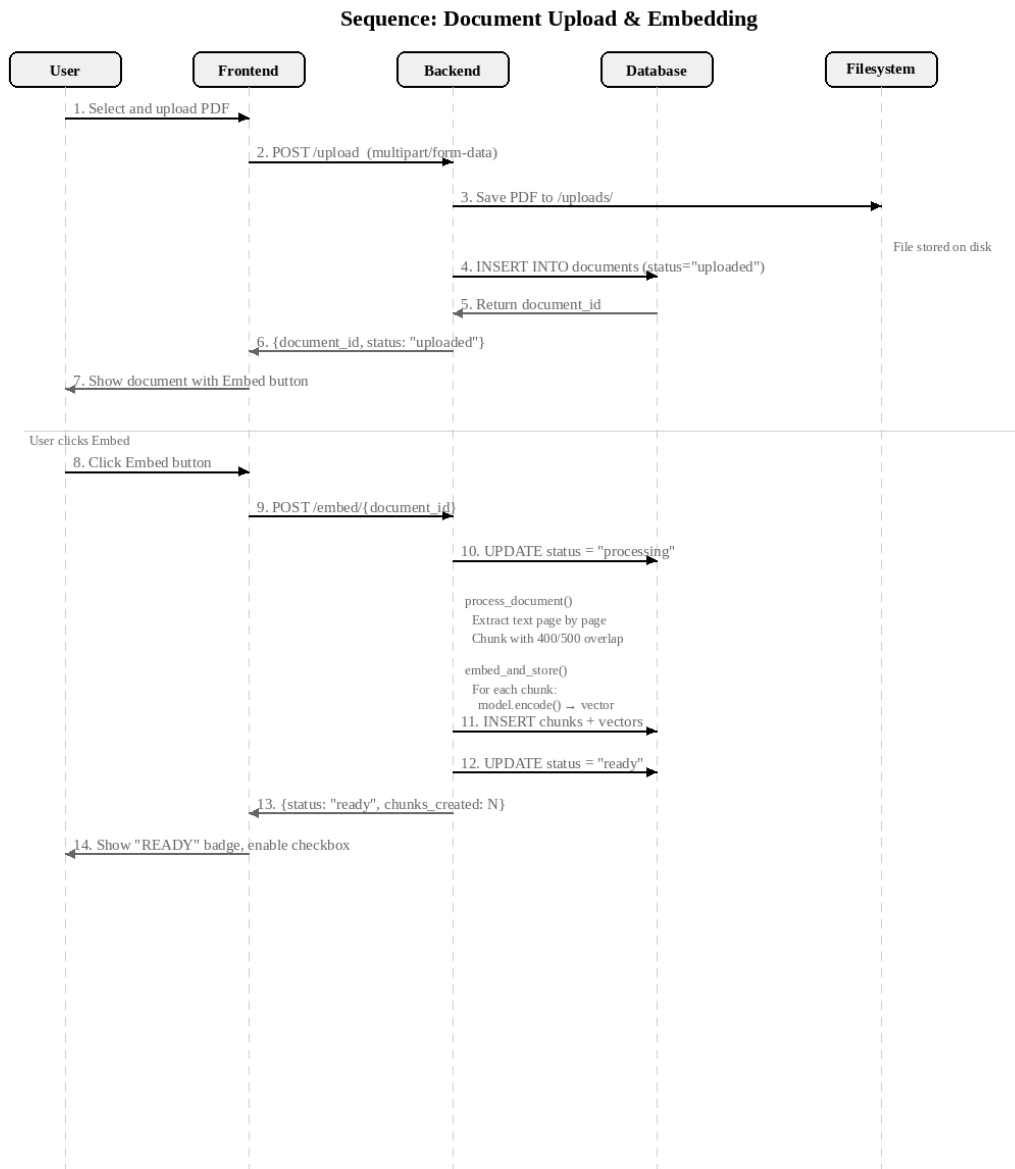
Process

- Embed the question using the same SentenceTransformer model
- Execute cosine similarity search against pgvector, filtered by selected document_ids
- Retrieve top 5 most similar chunks, ordered by descending similarity
- If no chunks found, return a fallback message
- Build a labeled context string: "Source 1 (Page N): [chunk text]" for each result
- Assemble the prompt with system message (constraints) and user message (question + context)
- Call Groq API and return the response text

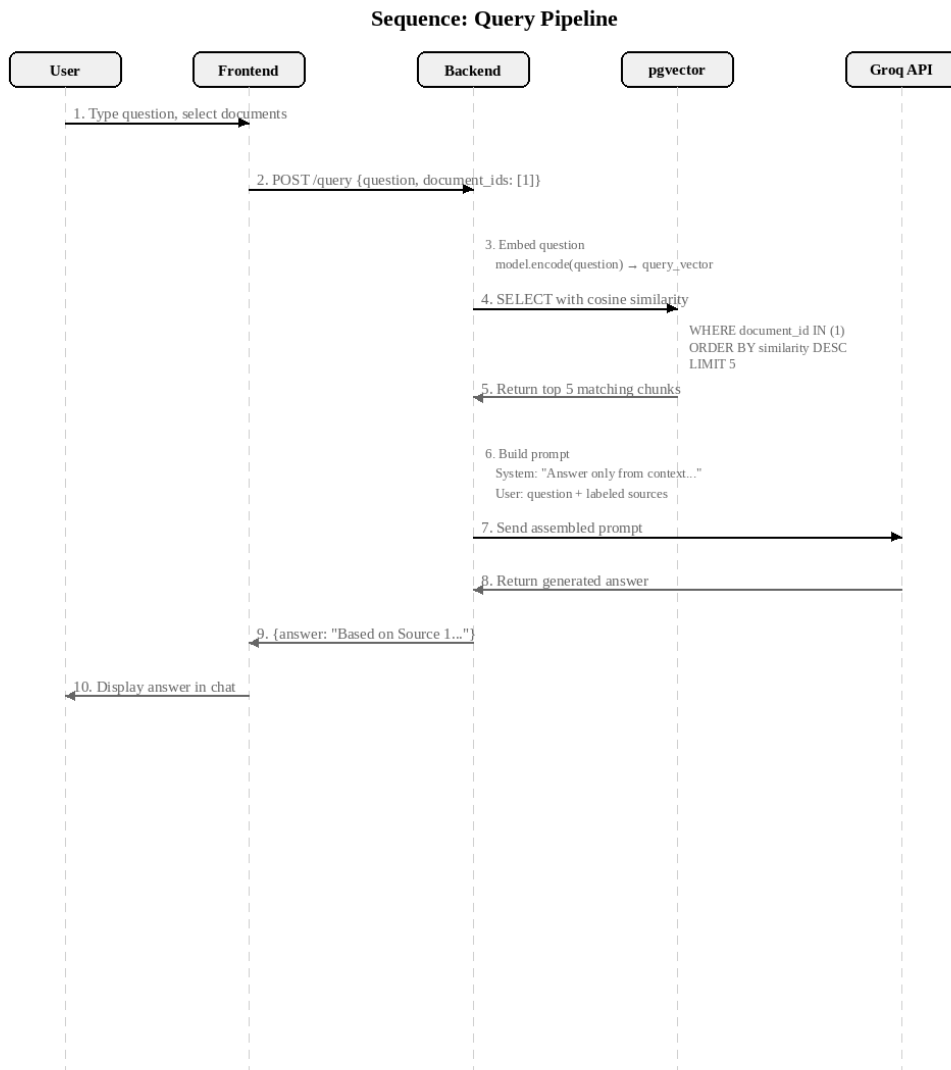
2.2 Key Design Decisions

- **Single embedding model instance:** Loaded once at startup to avoid reloading 90MB per request.
- **Overlapping chunks:** 100-char overlap prevents information loss at chunk boundaries.
- **Cosine similarity via pgvector:** The <=> operator computes cosine distance natively in PostgreSQL.
- **Prompt engineering:** System prompt constrains the LLM to answer only from provided context, reducing hallucination.
- **Synchronous embedding:** Appropriate for demo scope; production would use a background task queue.

2.3 Sequence Diagram: Document Upload & Embedding



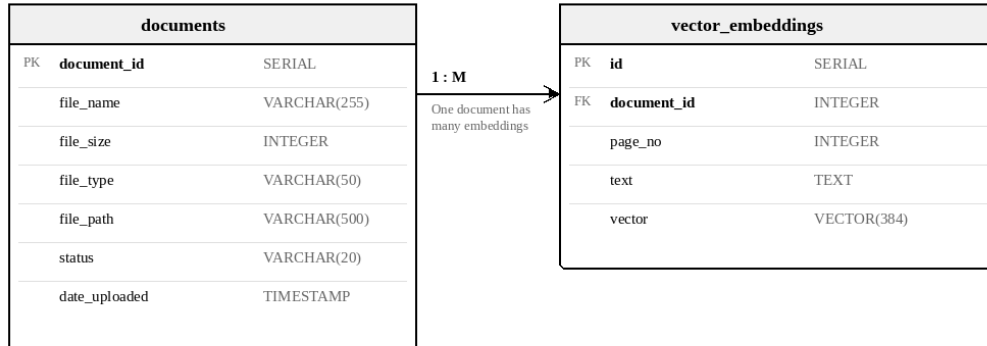
2.4 Sequence Diagram: Query Pipeline



3. Data Model / Database Schema (ERD)

3.1 Entity Relationship Diagram

Entity Relationship Diagram



3.2 Data Dictionary: documents

Column	Type	Null	Description
document_id	SERIAL (PK)	No	Auto-incremented primary key
file_name	VARCHAR(255)	No	Original filename of the uploaded PDF
file_size	INTEGER	Yes	File size in bytes
file_type	VARCHAR(50)	Yes	File extension (e.g., 'pdf')
file_path	VARCHAR(500)	No	Path to file on server filesystem
status	VARCHAR(20)	Yes	uploaded processing ready failed
date_uploaded	TIMESTAMP	Yes	Defaults to CURRENT_TIMESTAMP

3.3 Data Dictionary: vector_embeddings

Column	Type	Null	Description
id	SERIAL (PK)	No	Auto-incremented primary key
document_id	INTEGER (FK)	Yes	References documents.document_id
page_no	INTEGER	Yes	1-indexed page number of source chunk
text	TEXT	No	Original chunk text for LLM context
vector	VECTOR(384)	Yes	384-dim embedding from all-MiniLM-L6-v2

4. API Contracts / Interface Specs

Base URL: `http://localhost:8000`

Auto-generated OpenAPI documentation available at `/docs` (Swagger UI)

4.1 GET /health

Health check endpoint.

Field	Value
Method	GET
Response	<code>{"status": "ok"}</code>

4.2 POST /upload

Upload a PDF document. Saves file to disk and creates a metadata record.

Field	Value
Content-Type	multipart/form-data
Request Body	file: UploadFile (PDF only)
Success (200)	<code>{"document_id": 1, "file_name": "report.pdf", "status": "uploaded"}</code>
Error (400)	Non-PDF file uploaded

4.3 POST /embed/{document_id}

Trigger text extraction, chunking, and embedding for a document.

Field	Value
Path Parameter	document_id (integer)
Success (200)	<code>{"document_id": 1, "status": "ready", "chunks_created": 34}</code>
Error (404)	Document ID not found
Error (500)	Processing or embedding failure

4.4 GET /documents

Retrieve list of all documents with metadata and processing status.

Field	Value
Success (200)	<code>{"documents": [{document_id, file_name, status, ...}]}</code>

4.5 POST /query

Submit a question against selected documents. Returns LLM-generated answer.

Field	Value
-------	-------

Content-Type	application/json
Request Body	<code>{"question": "...", "document_ids": [1]}</code>
Success (200)	<code>{"answer": "Based on Source 1...", "question": "..."}"</code>
Error (400)	Empty document_ids list

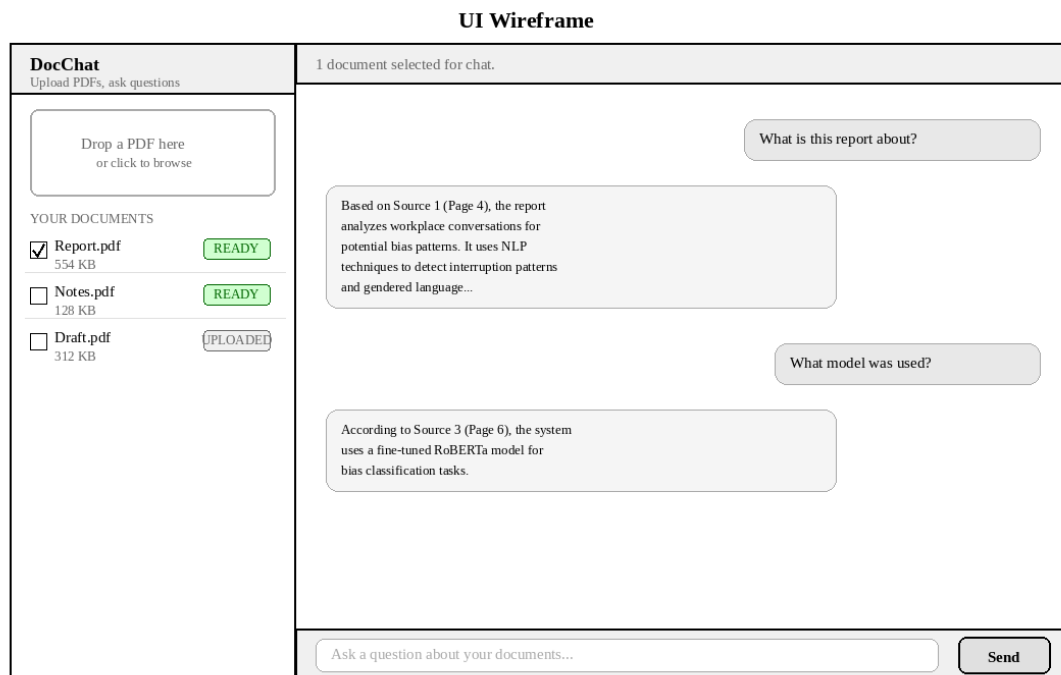
4.6 Error Response Format

All errors follow the standard FastAPI format: `{"detail": "error message"}`

Code	Meaning	When
400	Bad Request	Invalid file type, missing fields, empty document_ids
404	Not Found	Document ID does not exist
500	Server Error	PDF processing, embedding, or LLM API failure

5. UI/UX Design

5.1 Wireframe



5.2 Component Breakdown

Upload Dropzone

- Drag-and-drop zone with click-to-browse fallback
- Accepts only .pdf files; rejects others with error toast
- Visual feedback on dragover (border color change)

Document List

- Displays all documents sorted by upload date (newest first)
- Each document shows: filename, size, status badge, action button
- Status badges: UPLOADED (grey), PROCESSING (blue animated), READY (green), FAILED (red)
- Checkbox enabled only when status is READY

Chat Interface

- Header shows count of selected documents
- User messages right-aligned; assistant messages left-aligned
- Input bar with text field and Send button; supports Enter key
- Thinking indicator shown during LLM processing

Toast Notifications

- Bottom-right positioned; auto-dismiss after 4 seconds
- Three variants: info (blue), success (green), error (red)

5.3 Style Guide

Property	Value
Background (main)	#0f1117 (near-black)
Background (sidebar)	#161822 (dark charcoal)
Text (primary)	#e1e4e8 (light grey)
Accent	#4f8ff7 (blue)
Font family	Segoe UI, -apple-system, sans-serif
Border radius	8-10px for cards, 16px for chat bubbles